Name: Gurumanie Singh Dhiman
Section: A
University ID: 911192340

# Lab 4 Report

## Summary (10pts):

In this lab I learnt about inter-process communication (IPC) and how it works in UNIX, pipes, signals, message queues, shared memory and semaphores. I found out how SIGINT and SIGQUIT are generated and handled by the terminal and programs. We also explored how to override their behavior using the signal() function. In the pipe and shared memory sections of the lab, I learnt how the processes in the system exchange data and how important it is for us to synchronize so that we avoid any possible race conditions.

We also learned more about message queues and semaphores in terms of IPC. We also learnt on how these message queues and shared memory sections stay in the system until they are explicitly removed by a few different ways. We can remove them by either restarting the system, using msgctl() and shmctl(), or using commands like ipcrm in the terminal.

## Lab Questions:

### 3.1:

**6 pts** After reading through the man page on signals and studying the code, what happens in this program when you type CTRL-C? Why?

The integer value for CTRL-C was 2 and the value for CTRL-\ was 3. The reason why is that pressing CTRL-C sends the SIGINT (signal interrupt) signal, which has the integer value 2, and pressing CTRL-\ sends the SIGQUIT (signal quit) signal, which has the integer value 3.

**3 pts** Omit the signal(...) statement in main. Recompile and run the program. Type CTRL-C. Why did the program terminate?

As mentioned in part (a) the CTRL-C sends a SIGINT signal, the default action for this is to terminate the running process/program.

**3 pts** In main, replace the signal( ) statement with *signal(SIGINT, SIG_IGN)*. Recompile, and run the program then type CTRL-C. What's happening now?

This changes the CTRL-C behavior because we are overriding its behavior. It SIG_IGN is the ignore signal so the default behavior is being overriden to do nothing. This is why it doesn't work.

**3pts** The signal sent when CTRL-\ is pressed is SIGQUIT. Replace the *signal()* statement with *signal(SIGQUIT, my_routine)* and run the program. Type CTRL-\. Why can't you kill the process with CTRL-\ now?

Now we are overriding the CTRL-\ from its default behavior (of sending a SIGQUIT) to execute the function my routine instead.

## 3.2:

**5pts** What are the integer values of the two signals? What causes each signal to be sent?\

CTRL-C is value 2 and CTRL-\ is value 3. CTRL-C and CTRL-\ respectively are what cause the signal to be sent.

## 3.3:

**10 pts** Include your source code

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

// Signal handler
void handle_sigfpe(int signo) {
    printf("Caught a SIGFPE\n");
    exit(EXIT_FAILURE);  // Exit after we handle the signal
}

int main() {
    signal(SIGFPE, handle_sigfpe);
    int a = 4;
    int b = 0;
    // Create a division-by-zero scenario
    int c = a / b;  // This will cause SIGFPE
    printf("Result: %d\n", c);
    return 0;
}
```

**5pts** Explain which line should come first to trigger your signal handler: the signal() statement or the division-by-zero statement? Explain why.

Signal statement needs to go first because when flag SIGFPE is set it will run handle_sigfpe. If it isnt set before, the floating point exception might happen before the signal statement and the handler wont run.

## 3.4:

**4pts** What are the parameters input to this program, and how do they affect the program?

First parameter names the alarm and the second parameter is the time in seconds. They affect the program by setting the name and setting the time.

**6pts** What does the function “alarm” do?? Mention how signals are involved.

It sets an alarm. After a given number of seconds (that you input) it sends a SIGALARM signal to the program. This is then picked up by the signal function.

## 3.5:

**2pts** Include the output from the program.

```
bash-5.1$ ./3-5
Entering infinite loop
Entering infinite loop
^CReturn value from fork = 12317
Return value from fork = 0
```

**2pt** How many processes are running?

Two processes are running

**3pts** Identify which process sent each message.

The first “entering infinite loop” is sent by the Parent (main function) and the second is sent by the Child (child of main process).

**3pts** How many processes received signals?

Two processes received signals.

## 3.6:

**2pts** How many processes are running? Which is which (refer to the if/else block)?

There are a total of two processes that are running. According to the if/else block, the parent is the if block process that runs the write and the child is the else block that runs the read.

**6pts** Trace the steps the message takes before printing to the screen, from the array msg to the array inbuff, and identify which process is doing each step.

The parent process creates the pipe with two file descriptors p[0] for the read end and p[1] for the write end. Once that is done, they get inherited by the child process when the fork() is called. Since the msg “How are you?” is stored with the parent, the parent then writes to the pipe’s kernal buffer. The child process then calls the read and the OS transfers the msg from the pipe’s kernal buffer into the child (inbuff). Since the child’s inbuff now contains the msg, it is able to execute “How are you?” to the terminal output.

**2pts** Why is there a sleep statement? What would be a better statement to use instead of sleep (Refer to lab 2)?

The sleep statement is just there to delay the child process which gives the parent enough time to execute it’s write() before the child process tries to read(). Without the sleep statement, there would be a race condition where the child tries to read before the parent has done its write operation. It would be better to use a wait() in the parent process. This is what was used in lab2 and is a much better way to synchronize because sleep() might not work if our parent takes longer than the sleep statements time. Using wait() ensures that this does not occur and that the parent waits for the child to exit.

## 3.7:

**3pts** How do the separate processes locate the same memory space?

Server:

```
bash-5.1$ ./shm_server
Server detected client read
```

Client:

```
bash-5.1$ ./shm_client
Message read: abcdefghijklmnopqrstuvwxyz
Client done reading memory
```

The separate processes locate the same memory space by using the same shared memory key defined as key=5678. This key helps them uniquely identify their shared memory segment. The server creates the segment while the client connects to it. Basically both the processes attach using shmat to that same memory segment.

**3pts** There is a major flaw in these programs, what is it? (Hint: Think about the concerns we had with threads)

The major flaw in these programs is their lack of synchronization. The same as our readers writers problem, there are no semaphores or mutex locks to coordinate the read/write operations done. This can possibly lead to race conditions.

**3pts** Now run the client without the server. What do you observe? Why?

```
Message read: *bcdefghijklmnopqrstuvwxyz
Client done reading memory
```

The difference is that in this case the Message read is *bcdef… instead of abcd… in the first case. This is mainly due to the shared memory segment still existing and the program for our client code setting the * in the first byte so the next time we try to look at it, we get *bc… instead of abc… because our memory wasn't cleared from the last time we ran our code. This is the reason why we see *bc… instead of abc… since our previously run code modified the message.

**6pts** Now add the following two lines to the server program just before the exit at the end of main:
*shmdt(shm)*
*shmctl(shmid, IPC_RMID, 0)*
Recompile the server. Run the server and client together again. Now run the client without the server. What do you observe? What did the two added lines do?

```
bash-5.1$ ./shm_client
shmget failed: No such file or directory
```

Now with the two lines added, we get shmget failed. The shmdt(shm) detaches the memory segment and the shmctl() deletes the segment once all processes are detached from it. Effectively, the lines added basically detach and remove the shared memory segment from the system. This change is different from last time because as I mentioned, after our server and client were done running and we ran client on its own, our memory space still existed which is why we were able to see the modified *bc… instead of abc… but this time since we got rid of our shared memory segment with those two lines, we no longer have access to it.

## 3.8:

**2pts** Message queues allow for programs to synchronize their operations as well as transfer data. How much data can be sent in a single message using this mechanism?

```
bash-5.1$ cat /proc/sys/kernel/msgmax
8192
```

We can send 8KB (8192 bytes) in a single message using this mechanism. This max data value is set by the kernel parameter MSGMAX which can be found in /proc/sys/kernel/msgmax.

**2 pts** What will happen to a process that tries to read from a message queue that has no messages (hint: there is more than one possibility)?

There are two possible things that might occur. One possibility is that the process is put into a blocking mode which is the default behavior. Another possibility is that the process goes into a non-blocking mode which depends on if our call includes the IPC_NOWAIT flag. So in simple terms, our process either waits until a message is available which is the blocking mode or returns immediately with an error if we include the IPC_NOWAIT flag which is the non-blocking mode.

**3pt** Both Message Queues and Shared Memory create semi-permanent structures that are owned by the operating system (not owned by an individual process). Although not permanent like a file, they can remain on the system after a process exits. Describe how and when these structures can be destroyed.

Since both Message Queues and Shared Memory are managed by our kernel, they can be destroyed in three ways. One of them is what we used in our previous part (3.7) which was to use shmctl() for Shared Memory and another is to use msgctl() for removing Message Queues. The second way is to use commands ipcrm with the -m flag for shared memory or -q flag for message queues. The last way is to simply restart the entire system which will clear the Linux System's memory. As for the when, these structures can be destroyed if they are marked for deletion by the kernel, or when the last process detaches from that shared memory segment.

**3pt** Are the semaphores in Linux general or binary? Describe in brief how to acquire and initialize them.

The semaphores in Linux are general. To first acquire these semaphores we need to use the semget() with specifying the key which is the unique identifier and the nsems within the method which sets the number of semaphores in the set. To initialize these semaphores we need to use semctl() which requires specifying the value for the initial count. Finally with the above done we are now able to increment or decrement them using semop() with -semop(-1) as decrement and semop(+1) as increment.